OghmaNotes

AI-Powered Note-Taking with RAG & Canvas LMS Integration

CT216 — Software Engineering I

Project Report

Academic Year 2025/2026

Semyon Fox	24495896
Samuel Regan	24498764
Shreyansh Singh	24316243

Lecturer: **Enda Barrett**

University of Galway

Live Application: https://oghmanotes.ie
GitHub Repository: https://github.com/semyonfox/oghma

April 2026

Contents

1 Project Overview **1**
1.1 Problem Statement . . . 1
1.2 Our Solution . . . 1
1.3 Key Differentiators . . . 1

2 Team Roles & Contributions **1**
2.1 Semyon Fox — Tech Lead / Full-Stack / DevOps . . . 1
2.2 Samuel Regan — Authentication / RAG Pipeline . . . 2
2.3 Shreyansh Singh — Infrastructure / OAuth / Canvas API . . . 2

3 Customer-Facing Features **3**
3.1 Landing Page . . . 3
3.2 Authentication . . . 3
3.3 Notes Editor . . . 4
3.4 Canvas LMS Integration . . . 5
3.5 Search . . . 6
3.6 AI Chat (RAG-Powered) . . . 6
3.7 Settings . . . 6
3.8 Quiz System (Spaced Repetition) . . . 7
3.9 Productivity Tools . . . 7
3.10 Internationalisation . . . 8

4 Technical Architecture **8**
4.1 System Overview . . . 8
4.2 Technology Stack . . . 10
4.3 AWS Infrastructure . . . 10

5 REST API **10**
5.1 Authentication Middleware . . . 11

6 Key System Flows **11**
6.1 Authentication Flow . . . 11
6.2 Canvas Import Pipeline . . . 12
6.3 RAG Query Flow . . . 13

7 Database Design **13**
7.1 Schema Overview . . . 13
7.2 Vector Storage . . . 14

8 CI/CD & Deployment **14**
8.1 AWS Amplify Pipeline . . . 14
8.2 Branch Strategy . . . 14
8.3 Containerised Infrastructure . . . 14

9 Design Decisions & Architectural Pivots **15**
9.1 Pivot 1: Notea Fork to Modern Next.js Application . . . 15
9.2 Pivot 2: Monorepo to Single Application . . . 16
9.3 Pivot 3: BullMQ (Redis) to AWS SQS . . . 16

9.4 Pivot 4: Local Ollama to Cohere Embeddings . . . 16
9.5 Pivot 5: OpenWebUI as Secure Model Gateway . . . 16
9.6 Pivot 6: pnpm to npm . . . 17
9.7 Pivot 7: Cohere Embeddings Back to Self-Hosted Homelab . . . 17
9.8 Pivot 8: qwen3-embedding:8b to mxbai-embed-large (Right-Sizing for Hardware) . . . 17
9.9 Budget Constraints . . . 17

10 Testing **18**

11 Project Management **18**
11.1 Methodology . . . 18
11.2 GitHub Projects . . . 18
11.3 Timeline . . . 19
11.4 Commit Activity . . . 19

12 Challenges & Reflections **19**
12.1 Semyon Fox . . . 19
12.2 Samuel Regan . . . 20
12.3 Shreyansh Singh . . . 21

13 Future Work **21**

14 AI-Assisted Development Disclosure **22**
14.1 How AI Was Used . . . 22
14.2 What AI Did Not Do . . . 22
14.3 Reflection . . . 22

15 Resources & Links **23**

1 Project Overview

1.1 Problem Statement

University students' course content is scattered across LMS platforms, local folders, and personal notes. Tools like Notion and Obsidian have strong editors but treat uploaded documents as opaque files: users can't query their content semantically. Dedicated AI tools can answer questions but lack persistent knowledge bases scoped to a student's own materials. Nothing bridges the gap between a personal note-taking workspace and AI-powered retrieval over lecture content, integrated with the institution's LMS.

1.2 Our Solution

OghmaNotes is an AI-powered note-taking platform built for students. It gives users a single workspace to create and organise notes in a VSCode-inspired editor, import course materials from Canvas LMS, and query their entire knowledge base through a Retrieval-Augmented Generation (RAG) chat interface.

The platform is built with Next.js 16.2 on the frontend, a PostgreSQL database with the `pgvector` extension for vector similarity search, and a suite of AWS services (RDS, S3, SQS, ECS Fargate, SES, Amplify) for production-grade infrastructure. It is live at `https://oghmanotes.ie`.

1.3 Key Differentiators

What sets it apart: (1) RAG-powered search over notes and PDFs using semantic retrieval rather than keyword matching; (2) direct Canvas LMS integration with one-click course import handled by a self-scaling worker; and (3) a full AWS production stack built to real-world cloud engineering standards. The app also supports 12 languages through a custom internationalisation system.

2 Team Roles & Contributions

Three members, each owning distinct areas of the project, over seven months of development (September 2025 – April 2026).

2.1 Semyon Fox — Tech Lead / Full-Stack / DevOps

Led the technical side of the project: system architecture, infrastructure decisions, and the majority of feature development. Key contributions include:

- **Architecture & Infrastructure:** Designed and implemented AWS infrastructure including Amplify hosting, SQS job queues, ECS Fargate workers, and ElastiCache. Made all major architectural decisions including the BullMQ to SQS migration and embedding pipeline iterations.

- **Frontend:** Built the VSCode-style note editor UI, file tree, command palette, settings page, Canvas import UI, and AI chat interface. Implemented internationalisation across 12 languages.
- **Backend:** Developed API routes for notes CRUD, file upload, AI chat with RAG, and auth middleware with rate limiting.
- **DevOps:** Configured Amplify CI/CD pipeline, ECS Fargate worker deployment, and environment management across dev/main/production branches.
- **Canvas Integration:** Built the full Canvas LMS import pipeline from API integration through to S3 storage and RAG embedding.

2.2 Samuel Regan — Authentication / RAG Pipeline

Contributed to the authentication system, RAG pipeline, and frontend:

- **Authentication:** Built the login and registration UI, implemented password reset with Nodemailer (later migrated to AWS SES), and developed JWT-based session tokens with middleware-based route protection across the application.
- **Database:** Designed the original schema for the embeddings and chunks tables, later extended this to track worker state.
- **RAG Pipeline:** Implemented the full ingestion flow including PDF text extraction, text chunking, embedding generation, and pgvector similarity search. Designed an asynchronous job queue with a PostgreSQL jobs table to separate document ingestion from the upload request, avoiding timeout failures on large imports. The chat prompt construction and HNSW indexing were later extended by the rest of the team.
- **Frontend:** Contributed targeted fixes to the file rename and zoom functionality.

2.3 Shreyansh Singh — Infrastructure / OAuth / Canvas API

Contributed to the cloud infrastructure setup, authentication, and Canvas LMS integration:

- **Infrastructure:** Designed and provisioned core AWS services including RDS (PostgreSQL), S3 (object storage), and SES (transactional email). Configured security groups, IAM policies, and SSL certificates for database connectivity.
- **Database:** Designed the PostgreSQL schema under the `app` namespace (e.g. `app.login`, `app.notes`, `app.chunks`), making decisions around table structure, relationships, and the use of `pgvector` for embedding storage.
- **OAuth:** Implemented Google and GitHub OAuth flows using Auth.js (NextAuth), including provider registration, callback handling, and account linking logic.
- **Canvas API Integration:** Integrated the Canvas LMS REST API, covering course and file retrieval from university Canvas instances.

3 Customer-Facing Features

3.1 Landing Page

The landing page at `https://oghmanotes.ie` features a hero section, feature cards, team information, and authentication entry points. Built with Next.js App Router and Tailwind CSS v4.

Figure 1: Landing Page — Hero section with tagline and call-to-action

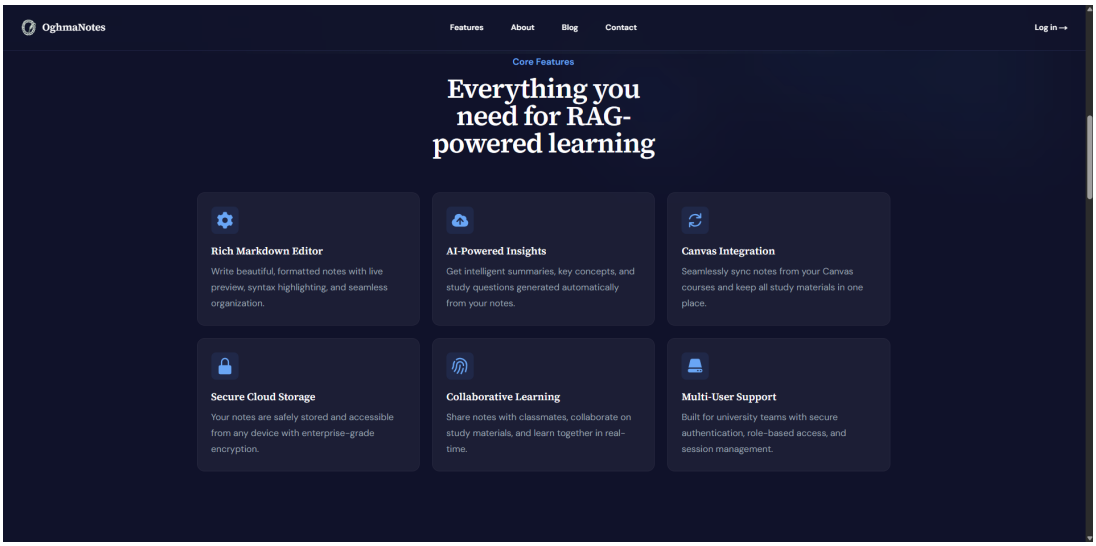

Figure 2: Landing Page — Core feature cards

3.2 Authentication

OghmaNotes supports email/password registration (with SES-based email verification and password reset) and OAuth 2.0 via Google and GitHub using Auth.js. Accounts

are deduplicated by email, and rate limiting is applied to all auth endpoints. The full authentication flow is detailed in Section 6.1.

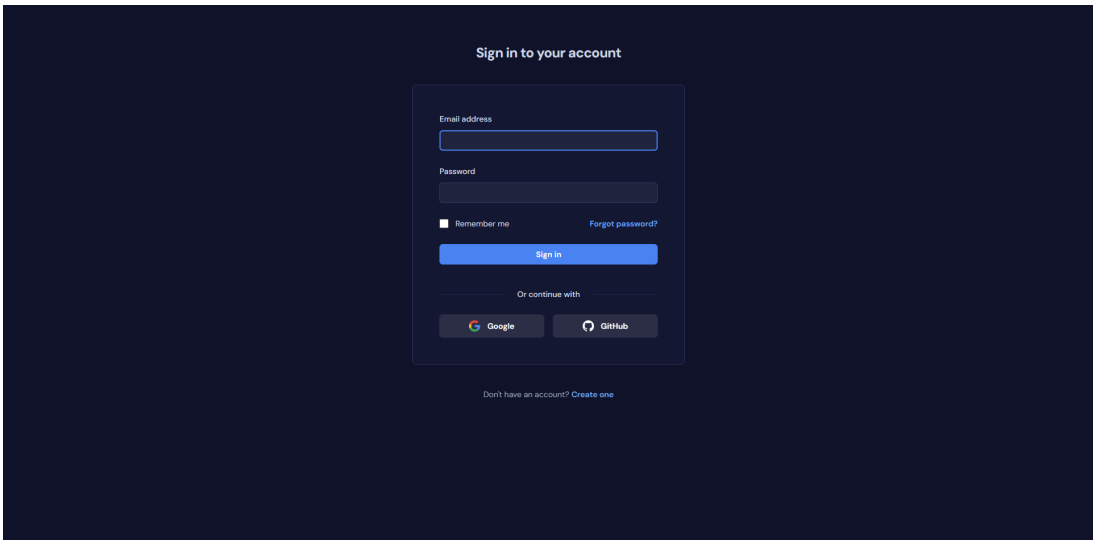

Figure 3: Login Page — Email/password form with Google and GitHub OAuth

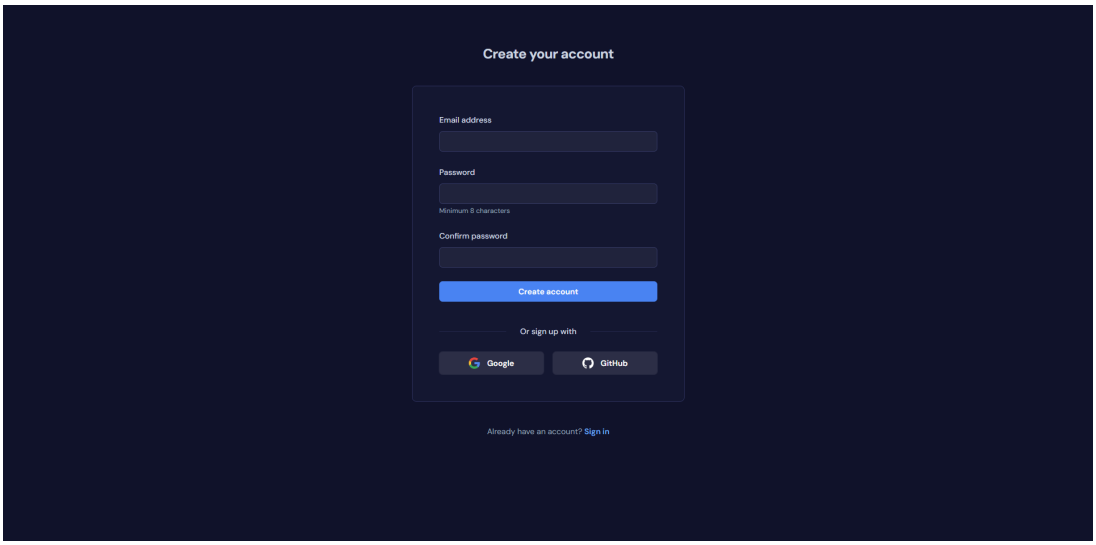

Figure 4: Registration Page — Account creation form with OAuth options

3.3 Notes Editor

The editor builds on the Notea foundation (S3-backed markdown storage), modernised and extended as described in Section 9.1. It uses a VSCode-inspired three-panel layout: a file tree sidebar (Obsidian-style with indent guides), a main editor pane (Lexical rich-text with CodeMirror for markdown source), and an optional split-pane PDF viewer for side-by-side study. File management covers create, rename, delete, duplicate, drag-and-drop upload, breadcrumb navigation, and a keyboard-accessible command palette.

Screenshot: Notes Editor — File tree sidebar with split-pane editor and PDF viewer

Figure 5: Notes Editor — File tree sidebar with split-pane editor and PDF viewer

Screenshot: Notes Editor — Command palette overlay

Figure 6: Notes Editor — Command palette overlay

3.4 Canvas LMS Integration

Users enter their Canvas API token and institution URL in settings. The token is encrypted at rest and decrypted at runtime for Canvas REST API calls. Users browse their courses and pick which to import; the import runs asynchronously via SQS and an ECS Fargate worker (see Section 6.2).

Screenshot: Canvas Import — Settings page with API token configuration

Figure 7: Canvas Import — Settings page with API token configuration

Screenshot: Canvas Import — Course selection and import progress

Figure 8: Canvas Import — Course selection and import progress

3.5 Search

Notes and imported documents are searchable via two complementary modes. Keyword search performs fast pattern matching against note titles and content. Semantic search embeds the query using `mxbai-embed-large` and retrieves the most relevant notes by cosine similarity via pgvector. Both modes support filtering by Canvas course, allowing students to scope searches to a specific module. The two modes run as separate requests and results are deduplicated client-side.

3.6 AI Chat (RAG-Powered)

Users ask natural-language questions about their notes and imported documents. The system embeds the query, retrieves relevant chunks via pgvector, reranks them, and streams a cited answer from the LLM. Search can be scoped to specific files or folders, sessions persist across visits, and responses can be rated with a thumbs up or down. The full pipeline is in Section 6.3.

Screenshot: AI Chat — Chat interface with RAG-powered responses and context attachments

Figure 9: AI Chat — Chat interface with RAG-powered responses and context attachments

3.7 Settings

The settings page provides profile management, dark mode, language selection (13 locales), Canvas API token configuration, OAuth account linking, and full vault export. The vault export streams all notes as Markdown files in a ZIP archive, preserving the folder hierarchy, with no proprietary format lock-in.

Screenshot: Settings Page — Profile, theme, language, and Canvas configuration

Figure 10: Settings Page — Profile, theme, language, and Canvas configuration

3.8 Quiz System (Spaced Repetition)

OghmaNotes includes a flashcard quiz system using the FSRS (Free Spaced Repetition Scheduler) algorithm via `ts-fsrs`. Users create flashcards from their notes and study with spaced-repetition scheduling tuned for long-term retention. The system tracks quiz sessions, individual card reviews, and streak data.

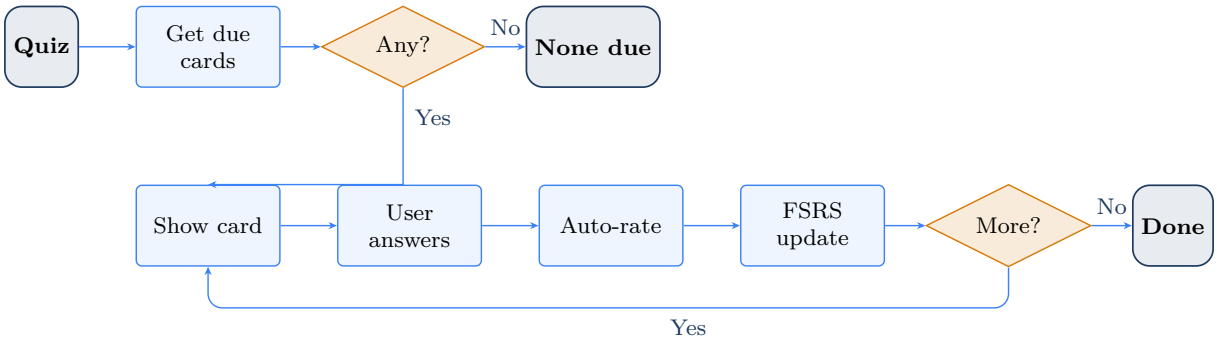

Figure 11: Quiz flow: FSRS card scheduling, difficulty rating, and interval update

3.9 Productivity Tools

Additional features include a Pomodoro timer with session tracking, a time-blocking calendar with Canvas assignment display and a subscribable iCal feed (authenticated via a per-user secret token stored in `app.login.calendar_export_token`, allowing direct subscription in any calendar client), and in-viewer PDF annotations.

Screenshot: Productivity Tools — Pomodoro timer, time blocks, and calendar view

Figure 12: Productivity Tools — Pomodoro timer, time blocks, and calendar view

3.10 Internationalisation

OghmaNotes supports 12 languages via a custom locale framework with JSON translation files. The active locale is stored in user preferences and applied across all UI strings.

4 Technical Architecture

4.1 System Overview

A serverless-first architecture on AWS with three runtime components: Next.js on Amplify, PostgreSQL (RDS with pgvector), and an ECS Fargate import worker, connected via S3, SQS, ElastiCache, and SES (Figure 13).

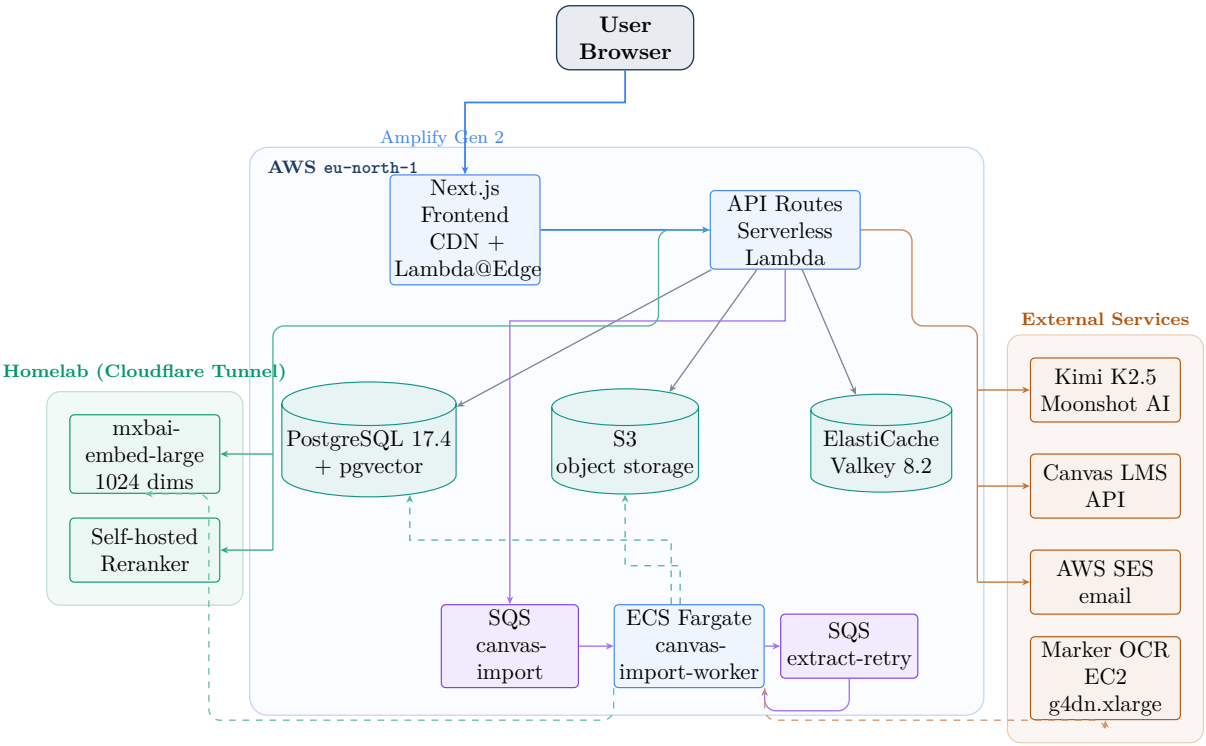

Figure 13: System Architecture — AWS infrastructure, homelab inference services, and external API dependencies

4.2 Technology Stack

Layer	Technology	Notes
Frontend	Next.js 16.2, React 19, TypeScript, Tailwind v4	VSCode-inspired split-pane editor
Backend	Next.js API Routes (serverless)	REST pattern with route handlers
Database	PostgreSQL 17.4 + pgvector (AWS RDS)	`app.*` schema, 8+ migrations
Object Storage	AWS S3	Notes, attachments, Canvas files
Cache	AWS ElastiCache (Valkey 8.2)	Session cache
Job Queue	AWS SQS	Canvas import job dispatch
Worker	AWS ECS Fargate	Self-scaling import worker
Email	AWS SES	Password reset, notifications
CI/CD	AWS Amplify Gen 2	Build, deploy, CDN, SSR Lambda
Auth	NextAuth v5 + Custom JWT	OAuth (Google/GitHub) + AES-256-GCM encryption
Embeddings	`mxbai-embed-large` (homelab, 1024 dims)	Self-hosted via Ollama; see Section 9.8 for migration history
Reranking	`Qwen3-Reranker-4B` (homelab, self-hosted)	Cross-attention reranking for RAG; Cohere deprecated
LLM	Kimi K2.5 (Moonshot AI)	Production LLM via OpenAI-compatible API
OCR	Marker (self-hosted EC2 ASG, g4dn.xlarge) + Datalab fallback	PDF/DOCX/PPTX text extraction
GPU Compute	AWS EC2 (g4dn.xlarge, NVIDIA T4, spot ~$0.35/hr)	Marker OCR via ASG, scales to zero
LMS	Canvas LMS REST API	Course/file import pipeline
i18n	Custom locale system	12 languages

Table 1: OghmaNotes Technology Stack

4.3 AWS Infrastructure

Production runs in `eu-north-1` (Stockholm). Table 1 lists each service. Key details: RDS uses a `db.t3.micro` with SSL, all tables under the `app` schema; S3 uses presigned URLs for client-side uploads; SQS decouples the API from the long-running Canvas import worker; ECS Fargate scales to zero when idle; a `g4dn.xlarge` GPU instance (NVIDIA T4, spot pricing) hosts the Marker OCR server behind an Auto Scaling Group that also scales to zero, with the Datalab Convert API as fallback.

5 REST API

The backend exposes 16 REST API route groups under `/api/` via Next.js API Routes, covering authentication, notes CRUD, file operations, Canvas import management, AI chat, search, quiz/flashcard management, productivity features, and data portability.

5.1 Authentication Middleware

All protected routes use a custom JWT middleware that extracts the token from HTTP-only cookies, verifies the signature, and attaches the decoded user to the request context. Unauthenticated requests get a 401. Rate limiting uses a sliding-window counter, with tighter limits on auth endpoints (login, register, password reset) to block brute-force attempts.

Route Group	Scope
`/api/auth/*`	Registration, login, logout, password reset, session
`/api/notes/*`	Notes CRUD and metadata
`/api/tree`	Folder hierarchy and drag-and-drop ordering
`/api/upload/*`	File upload to S3, presigned URL retrieval
`/api/extract`	PDF text extraction via Marker OCR
`/api/search`	Combined fuzzy + semantic search
`/api/chat/*`	RAG chat and session management
`/api/canvas/*`	Canvas connection, course listing, import, status
`/api/quiz/*`	Quiz sessions, answers, flashcards, streaks, dashboard
`/api/pomodoro`	Pomodoro timer sessions
`/api/time-blocks`	Time block calendar management
`/api/settings`	User preferences and account management
`/api/annotations`	PDF annotation storage
`/api/calendar`	iCal feed export (per-user secret token)
`/api/export`	Vault ZIP export
`/api/embeddings`	Embedding pipeline status and ingestion management

Table 2: REST API Route Groups (16 total)

6 Key System Flows

6.1 Authentication Flow

Three paths converge on a shared JWT session: email/password login (bcrypt), registration (SES email verification), and OAuth via Google/GitHub (Auth.js). All paths issue an HTTP-only JWT cookie encrypted with AES-256-GCM. Token refresh goes through `/api/auth/me`.

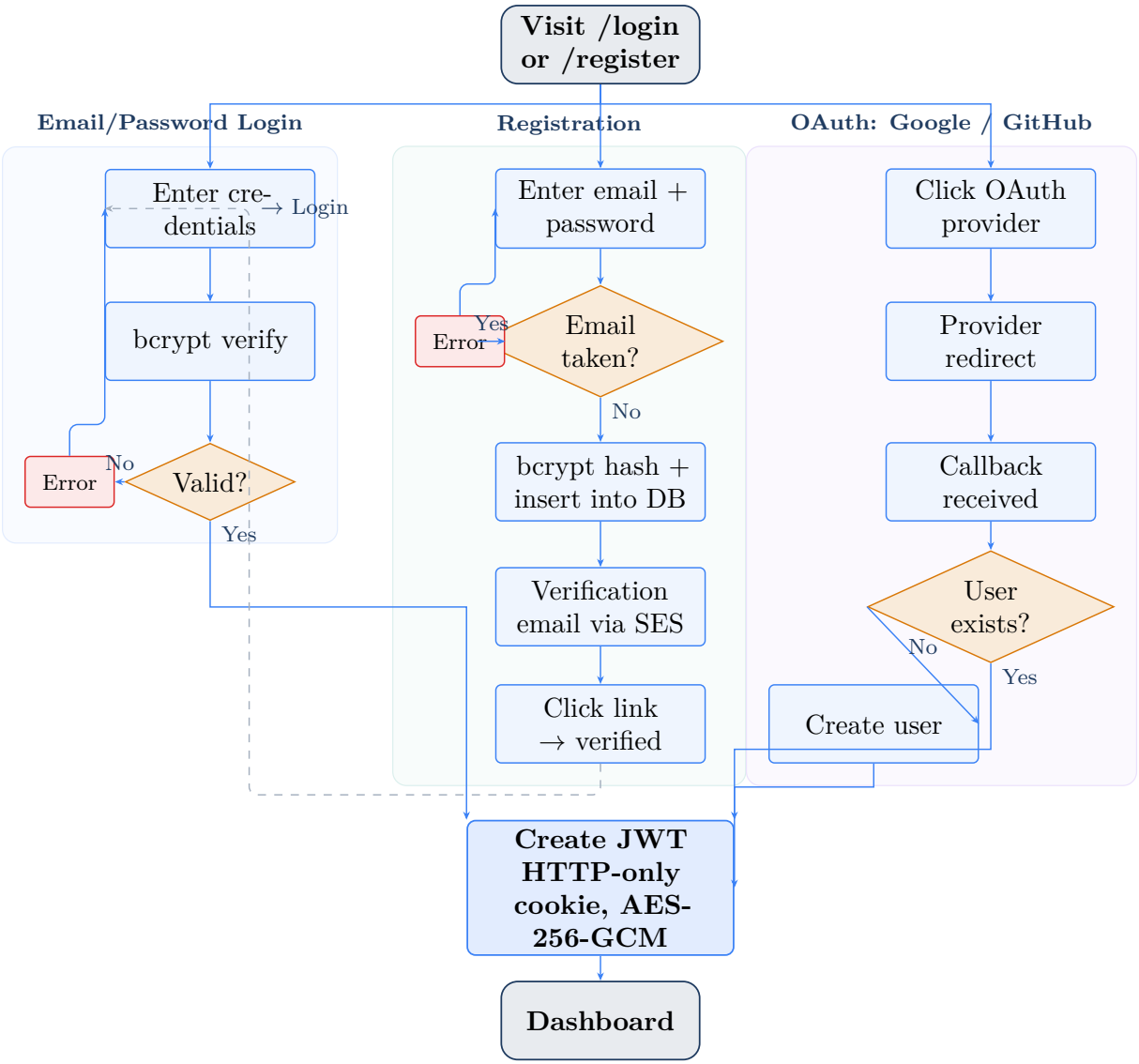

Figure 14: Authentication Flow — Email/password, registration with email verification, and OAuth paths

6.2 Canvas Import Pipeline

The Canvas import touches five AWS services. The API creates a job record and publishes to SQS. An ECS Fargate worker picks it up and runs: Canvas API file retrieval → S3 upload → Marker OCR text extraction (Datalab fallback) → sentence-aligned chunking (∼500 chars) → embedding via self-hosted model → pgvector storage. The worker scales to zero after an idle timeout.

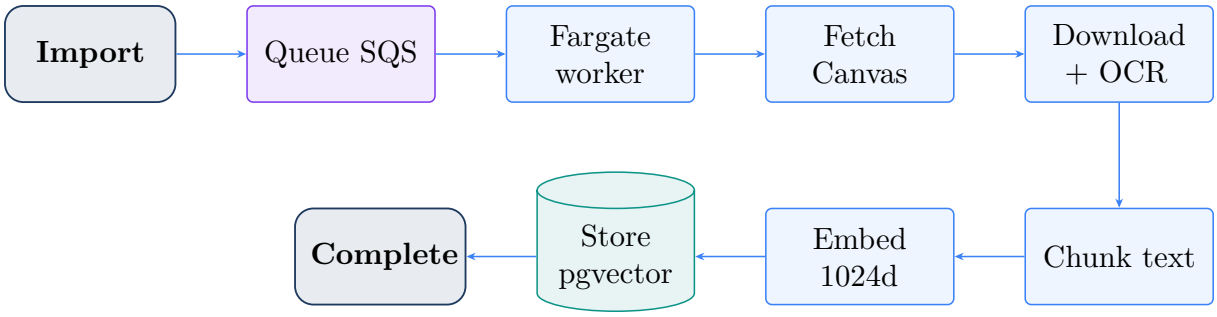

Figure 15: Canvas Import Pipeline — SQS job dispatch, ECS Fargate worker processing, and pgvector storage

6.3 RAG Query Flow

The query is embedded (self-hosted `mxbai-embed-large`, 1024d), searched against stored chunks via pgvector cosine similarity (top-20 within distance 0.75, optionally scoped to selected files), reranked by a self-hosted cross-attention model (top-5 above a 0.15 relevance floor), injected into a prompt, and streamed to the user from Kimi K2.5 via SSE. Messages and responses are persisted for session continuity. Thresholds were tuned empirically against university lecture queries.

Figure 16: RAG Query Pipeline — Query embedding, pgvector cosine search, self-hosted reranking, and LLM streaming

7 Database Design

7.1 Schema Overview

PostgreSQL 17.4 with `pgvector`, hosted on AWS RDS. All 19 tables sit under the `app` schema. Core tables: `app.login` (users), `app.notes`/`app.tree_items` (content and file hierarchy), `app.chunks`/`app.embeddings` (RAG vectors), and `app.chat_sessions`/`app.chat_messages` (conversations). Supporting tables cover OAuth linking, Canvas imports, quiz flashcards, Pomodoro sessions, time blocks, PDF annotations, and rate limiting. The schema went through 23 tracked versions in `app.schema_migrations`, covering the embedding dimension change (Section 9.8), two-phase Canvas import, rate limit audit logging, quiz infrastructure, chat message rating, and per-user calendar export tokens.

Screenshot: Entity-Relationship Diagram — Database schema relationships

Figure 17: Entity-Relationship Diagram — Database schema relationships

7.2 Vector Storage

Each chunk's 1024-dimensional embedding is stored via pgvector with HNSW indexing for approximate nearest-neighbour search. The embedding column was migrated from `vector(4096)` to `vector(1024)` as part of Section 9.8, with all content re-embedded (see Section 9).

8 CI/CD & Deployment

8.1 AWS Amplify Pipeline

The project uses AWS Amplify Gen 2 for CI/CD. Every push to `main` triggers the pipeline, which runs through three stages:

1. **Install:** Runs `npm ci` to install dependencies from the lockfile.
2. **Build:** Executes `next build`, producing optimised static assets and serverless functions.
3. **Deploy:** Distributes the build output to Amplify's CDN and deploys SSR routes as Lambda@Edge functions.

Environment variables (database credentials, API keys, AWS service endpoints) are managed through Amplify's environment configuration, with separate sets for the `dev` and `main` branches.

8.2 Branch Strategy

The repository uses two branches. `dev` is where features are built and tested. When stable, they merge to `main` via pull request, which triggers the production deployment. Each branch has its own Amplify environment with an independent configuration.

8.3 Containerised Infrastructure

The repository has three Dockerfiles, each for a distinct runtime: the Next.js production app (three-stage Alpine build with standalone output), the Canvas import

worker (`Dockerfile.worker`, Node 22 with `tsx`), and the Marker OCR service (`Dockerfile.marker`, Python 3.11 with GPU support). Worker and Marker images are pushed to Amazon ECR; ECS Fargate pulls the worker image on demand, scaling from zero when SQS messages arrive.

A Husky v9 pre-commit hook enforces quality gates on every commit: full ESLint linting and the Vitest test suite (blocking), `Dockerfile.marker` syntax validation via `docker build -dry-run`, and `shellcheck` on infrastructure shell scripts. For the Marker OCR instance, a custom AMI baking script (`bake-ami.sh`) pre-installs Python dependencies and the CUDA runtime into a machine image, cutting cold-start time from 5–7 minutes to around 90 seconds. The baked AMI is deployed via a Launch Template inside an Auto Scaling Group behind an Application Load Balancer, with IAM roles granting ECR pull access.

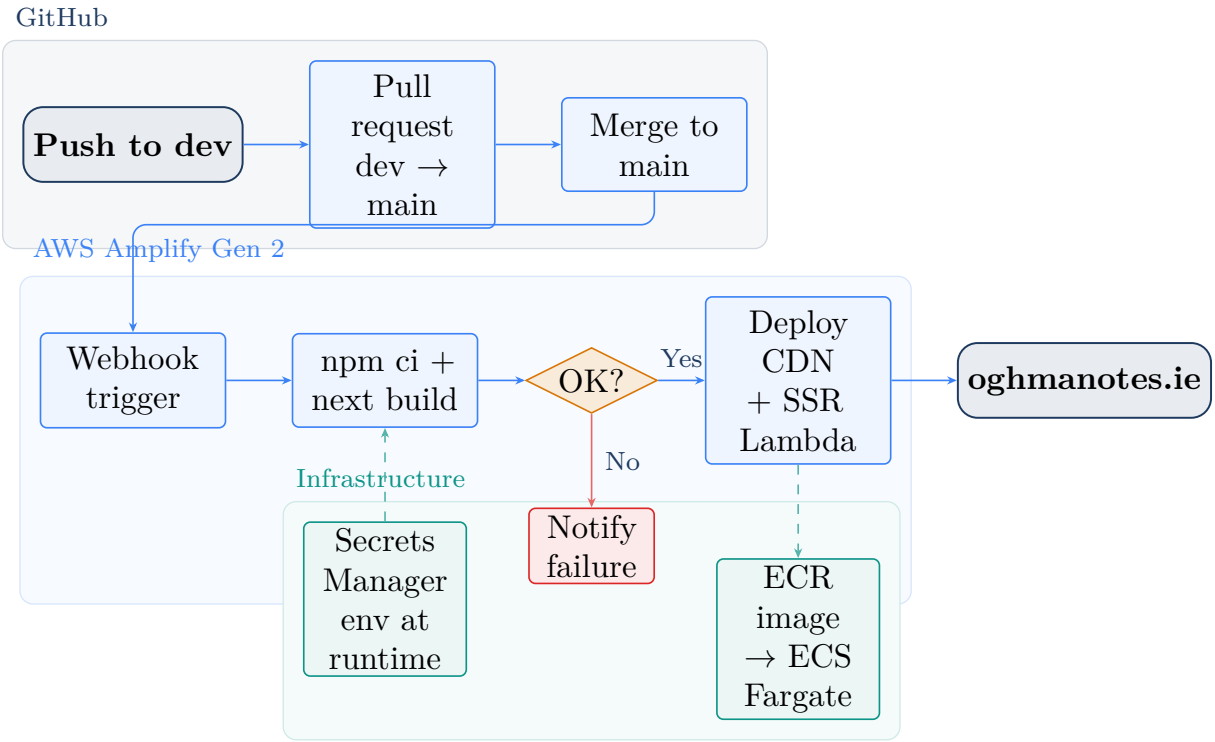

Figure 18: CI/CD Pipeline — GitHub push to AWS Amplify build, deploy, and infrastructure update

9 Design Decisions & Architectural Pivots

The team made several architectural pivots during development, each driven by constraints hit at runtime.

9.1 Pivot 1: Notea Fork to Modern Next.js Application

The project started from Notea, an MIT-licensed, unmaintained note-taking app that stored markdown on S3 — a useful backbone to scaffold from. Notea's codebase ran on outdated dependencies throughout the stack. Modernising it meant upgrading through

three major React versions (16 → 19), where APIs were deprecated or removed, replacing the legacy editor with Lexical and CodeMirror, and swapping unmaintained packages for supported alternatives. That upgrade work was heavy, but it left the team with a production-ready S3-backed markdown foundation to build RAG, Canvas integration, and the full AWS infrastructure on top of.

Lesson: Forking saves initial scaffolding time, but inheriting an unmaintained dependency tree creates upgrade debt that compounds with each major framework version.

9.2 Pivot 2: Monorepo to Single Application

The project started as a monorepo with separate packages under an `apps/web/` directory. The added complexity in Amplify's build configuration wasn't worth it for a three-person team. Collapsing to a single Next.js application simplified both development and deployment.

9.3 Pivot 3: BullMQ (Redis) to AWS SQS

The Canvas import job queue started as BullMQ backed by ElastiCache (Valkey/Redis). It worked locally but broke in production: Amplify's Lambda functions couldn't reach ElastiCache without VPC configuration, and the Lambda runtime's 29-second timeout was incompatible with long-running import jobs. Migrating to AWS SQS fixed both problems. SQS uses IAM authentication (no VPC needed) and fully decouples the API from the worker.

Lesson: Cloud networking constraints shape architecture more than theoretical best practices. SQS was the pragmatic choice given the deployment environment.

9.4 Pivot 4: Local Ollama to Cohere Embeddings

The initial embedding pipeline used a self-hosted Ollama instance running `qwen3-embedding:8b`, served over a Cloudflare Tunnel. The model was fine; the tunnel wasn't. Embedding requests during Canvas imports would intermittently time out, causing ingestion jobs to fail mid-batch. Rather than debug the tunnel, the team migrated to Cohere's `embed-multilingual-v3.0` API, which gave a stable endpoint with consistent latency. Cohere's 1024-dimensional output was also compatible with pgvector's HNSW index limit.

Lesson: Infrastructure reliability is as important as model quality. A locally-hosted model routed through a tunnelled connection introduces failure modes that a direct API call does not.

9.5 Pivot 5: OpenWebUI as Secure Model Gateway

The homelab models (embedding, reranking) sit behind OpenWebUI, which handles auth and exposes them over a Cloudflare Tunnel with sign-ups disabled. Kimi K2.5 is called directly via its OpenAI-compatible API. Switching between self-hosted and cloud providers is a single environment variable change.

9.6 Pivot 6: pnpm to npm

A mid-project switch from pnpm to npm was forced by compatibility issues with the Amplify build environment. pnpm's workspace features conflicted with Amplify's build process. Switching to npm resolved the build failures with minimal disruption.

9.7 Pivot 7: Cohere Embeddings Back to Self-Hosted Homelab

After migrating to Cohere (Section 9.4), the team hit aggressive rate limiting on Cohere's free-tier embedding API during Canvas imports, where hundreds of chunks needed embedding in quick succession. Rather than pay for a higher tier, the team fixed the underlying Cloudflare Tunnel reliability issues and moved back to the self-hosted `qwen3-embedding:8b` model on the project lead's homelab. That eliminated rate limits and external API costs. With the tunnel stable, the team also restored the original 4096-dimensional vectors for better semantic fidelity, accepting sequential pgvector scanning at current data scale. The reranker moved to self-hosted infrastructure at the same time.

Lesson: External API dependencies introduce constraints beyond cost. Rate limits can silently degrade user experience. Self-hosting critical pipeline components gives control over throughput at the cost of operational responsibility.

9.8 Pivot 8: qwen3-embedding:8b to mxbai-embed-large (Right-Sizing for Hardware)

After restoring self-hosted embeddings (Section 9.7), the team hit a hardware bottleneck. The `qwen3-embedding:8b` model needed 6.1 GB of memory, but the server's GPU (a GTX 1050 Ti Max-Q) had only 4 GB of VRAM. Ollama split the model automatically: 59% on GPU, 41% falling back to CPU on the i7-8750H. That CPU portion became the dominant bottleneck, causing erratic latency that varied with whatever else the CPU was doing.

Running it entirely on CPU would have been worse. Instead, the team switched to `mxbai-embed-large`, a 335M-parameter model that fits within the 4 GB VRAM budget. The migration required changing the schema from `vector(4096)` to `vector(1024)`, clearing all existing embeddings, and re-embedding everything. A secondary benefit: 1024 dimensions falls within pgvector's HNSW index limit of 2000, enabling approximate nearest-neighbour indexing that wasn't possible at 4096 dimensions.

Lesson: Model selection needs to account for the full inference environment, not just benchmark quality. A smaller model running entirely on GPU will consistently outperform a larger one partially offloaded to CPU. Right-sizing to the available hardware eliminated the erratic latency and improved embedding throughput by roughly 10×.

9.9 Budget Constraints

The project ran on AWS education credits. That budget directly shaped infrastructure decisions: the self-scaling ECS Fargate pattern (scaling to zero when idle) was chosen to keep compute costs low. Instance sizes were kept minimal (`db.t3.micro` for RDS, `cache.t3.micro` for ElastiCache), and everything was deployed to a single region (`eu-north-1`) to avoid cross-region transfer costs.

10 Testing

The project uses Vitest as its test framework. The test suite comprises 52 test files covering: authentication (JWT, bcrypt, OAuth account linking, login lockout), input validation, rate limiting (Redis sliding-window with in-memory fallback), the FSRS spaced repetition card state machine, quiz generation and Bloom's taxonomy ZPD tracking, Canvas assignment sync, RAG pipeline utilities (chunk normalisation, SSE parsing, LLM stream handling, reranking), embedding batch client, text chunking, cryptographic token generation, tree actions, SQS integration, OCR output parsing, calendar date handling, note routing, vault tree building, API endpoint contracts (notes, settings), and AI configuration env parsing. The Husky pre-commit hook runs the full suite on every commit, blocking merges on test failure.

Coverage is configured via `@vitest/coverage-v8` with text, HTML, and LCOV reporters, with `src/__tests__` excluded from the measured surface. A dedicated `npm run test:coverage` script produces per-file coverage reports.

End-to-end testing was done manually across the full user journey, and classmates were invited to use the live application for informal user-acceptance feedback. Automated frontend and pipeline test coverage is a known gap. The BullMQ-to-SQS migration failure, which only showed up in production, is a concrete example of where broader integration tests would have helped.

11 Project Management

11.1 Methodology

The team used an agile-inspired approach without locking into any specific framework. Work ran in roughly two-week sprints, with priorities set at the start of each sprint based on feature dependencies and upcoming deadlines. Communication happened mainly through GitHub Issues and group messaging.

11.2 GitHub Projects

Task tracking ran through GitHub Issues and Pull Requests. The repository accumulated 117 issues (96 closed, 21 open) and 54 pull requests over the project lifetime. Issues were categorised by feature area (authentication, editor, Canvas integration, RAG, infrastructure) and assigned to team members. Pull requests required code review before merging to `main`, with notable PRs including security hardening (PR #155), the quiz system (PR #135), and a batch sprint (PR #133 resolving 37 issues).

Open issues at project close include centralising RAG pipeline constants, converting worker files to TypeScript, adding CloudWatch metrics for the embedding pipeline, and addressing remaining security advisories. An honest backlog of known technical debt.

Screenshot: GitHub Issues — Issue tracker showing feature categorisation and resolution

Figure 19: GitHub Issues — Issue tracker showing feature categorisation and resolution

11.3 Timeline

Period	Phase	Key Activities
Sep 2025	Planning	Initial commit, Next.js setup, documentation
Oct–Nov 2025	Semester 1 Dev	Login prototype, auth system, Docker setup
Dec 2025	Semester 1 Wrap	API fixes, deployment debugging, schema work
Jan 2026	Semester 2 Start	Fresh Next.js init, Tailwind templates, restructure
Feb 2026	Core Build	Amplify deployment, S3 storage, editor redesign, auth, RAG, Canvas research
Mar 1–10	Feature Sprint	Canvas import, RAG merge, SQS migration, Fargate, Cohere, chat, i18n
Mar 15–25	Polish & Fix	PDF viewer, DB consolidation, domain migration, SES, translations, UI audit
Mar 26 – Apr 3	Report & Hardening	Documentation, report writing, GDPR account deletion, Marker ASG deployment, ingestion queue refactor, frontend unification

Table 3: Development Timeline

11.4 Commit Activity

Most of the work happened in February and March 2026, with the RAG pipeline, Canvas import worker, and quiz system all landing within a six-week window. Earlier months were spent on infrastructure setup and the bigger architectural decisions; the final weeks were mostly hardening, docs, and compliance. Dependabot kept a trickle of dependency updates coming throughout.

12 Challenges & Reflections

12.1 Semyon Fox

As technical lead, my biggest challenge was scope management. The combination of AI-assisted development and genuine ambition created a feedback loop: every feature felt

achievable, so we kept adding them. The project grew from a simple notes app to a full-stack platform with RAG, Canvas integration, spaced repetition, and a GPU-backed OCR pipeline — all within a university project timeline.

The most technically demanding work was the AWS infrastructure. Debugging why Amplify's Lambda functions couldn't reach ElastiCache (VPC networking), why SQS messages weren't being consumed (IAM permissions), and why Marker's GPU instance took 15 minutes to cold-start (pip installing marker-pdf on every boot, solved by pre-baking a Docker image) each consumed entire days. These problems don't appear in tutorials or documentation — they emerge from the specific intersection of services in your architecture.

My role was more glue than solo builder. Major architectural decisions were discussed as a group, and AI-assisted development meant features that would otherwise have taken weeks could be scoped and prototyped quickly. That speed was a double-edged sword — it made the project's ambition feel achievable but compressed the time available for the kind of documentation and interface contracts that would have made integration smoother for everyone.

The key lesson from this project is that architecture decisions made under time pressure have long-lasting consequences. The BullMQ-to-SQS migration, the embedding pipeline iterations (Ollama to Cohere to self-hosted), and the monorepo collapse each cost days of rework. Building a proof-of-concept for each integration point before committing to it would have saved significant time.

12.2 Samuel Regan

For me, the most glaring challenge wasn't writing code. It was the learning curve and the invisible layer between the code and the services.

In a project of this magnitude, there were so many components from the AWS services such as instances and environment variables that when something failed even getting the logs for the deployed project was a huge endeavour. Most of the time the code would fail silently and it was almost always not an internal server error, but a configuration issue. For example, I spent an entire day debugging why the rag pipeline worker queue was not dispatching ingestion requests, and when it did it couldn't find the database or persist any data. The root cause was a second database instance running in parallel. My IDE was pointed to a different, older one, so the SELECT checks always returned empty chunk columns.

The ambition for this project between myself and my teammates compounded this. This was our first chance to develop a real full stack app alongside our course, and thus we had chosen the technologies and methodologies we believed would be most suitable for the project and consequently, I was overwhelmed with a long list of technologies, such as TypeScript, Next.js, and a full array of AWS services, as well as learning team Git workflows for the first time which all formed into a very harsh learning curve.

Upon reflection, I have learnt that on a production level, programming skill and familiarities with the technologies are equally important. For the first half of this project I neglected the services making the production possible and focused on the backend code, and that tunnel vision backfired greatly when I tried to integrate my features into the

rest of the system. That said, thanks to this ambitious project I have become far more familiar with cybersecurity, the connection between frontend and backend functions/services such as features being too slow for a UI facing application,integrating LLM models and manipulating them for niche purposes, the services required to host an application on AWS, and the critical thinking required to invent my own solutions to problems, such as our RAG pipeline ingesting too slowly. The very worker queue I described debugging earlier was my solution to that problem, processing documents in an ordered queue with workers rather than piling on large datasets to be ingested at once.

12.3 Shreyansh Singh

Working on OghmaNotes was my first experience with production cloud infrastructure. Setting up AWS RDS with SSL certificates, configuring S3 bucket policies, and getting SES verified for transactional email each presented their own learning curves. The most challenging aspect was understanding how the various AWS services connect — security groups, IAM roles, and VPC configurations are not intuitive when coming from a local development background.

The OAuth implementation with Auth.js required careful attention to the callback flow and account linking logic, particularly handling edge cases where a user might register with email and later try to sign in with Google using the same email address. The Canvas API integration involved navigating a large REST API surface and understanding the Canvas data model (courses, modules, items, files), which required significant research and experimentation.

The key takeaway from this project was that real-world software engineering is as much about infrastructure and integration as it is about writing application code. The skills I developed in AWS, database design, and API integration are directly applicable to professional work.

13 Future Work

Hybrid Search: Combining vector similarity with BM25 keyword search would improve retrieval for queries with technical terms or proper nouns, where exact matching helps alongside semantic understanding.

Note Tagging and Bidirectional Linking: Arbitrary tags and wiki-style `[[Note Title]]` links with backlink display and a knowledge graph would improve note discoverability and reflect the original SRS scope.

Analytics Dashboard: A dashboard showing study time trends, quiz score history, and weak topic identification would give students more insight into their learning patterns. The current quiz dashboard has a summary widget (streak, mastery percentage, cards due today); full time-series analytics are a planned next step.

Auto-Scheduling Study Sessions: Auto-generating time blocks from Canvas assignment due dates and FSRS review schedules would close the loop between the LMS integration and the productivity tools.

Collaborative Editing: Real-time collaborative editing would let study groups work on shared notes simultaneously, with the RAG system querying the shared content.

Expanded LMS Support: Integrations for Moodle, Blackboard, and other LMS platforms would extend the app beyond Canvas.

BYOK (Bring Your Own Key): Allowing students to connect their own LLM API keys would provide model choice and reduce platform costs. The settings UI stub already exists.

Offline Support (PWA): A Progressive Web App with IndexedDB caching would let students review notes and flashcards without a connection, syncing when they're back online.

14 AI-Assisted Development Disclosure

This project used AI coding assistants throughout development, primarily Claude Code (Anthropic) with some use of OpenCode and Codex. This section describes how the team worked with these tools.

14.1 How AI Was Used

AI was used as an implementation accelerator. Developers defined all architectural decisions, feature priorities, and design constraints; AI tools generated code against those specs, which was then reviewed, tested, and adjusted before committing. Common uses included boilerplate for repetitive patterns (API routes, database migrations, i18n translation files), working through AWS configuration issues, and UI refinement.

14.2 What AI Did Not Do

AI did not make architectural decisions. Every pivot in Section 9, from VPC networking failures to embedding model right-sizing, was diagnosed by the developers through production logs, hardware profiling, and AWS documentation. AI generated code; humans debugged it, fixed what broke in production, and made the judgment calls that shaped the system. Diagnosing failures AI couldn't see (cloud networking, GPU memory splits, rate-limit edge cases) turned out to be the most valuable learning experience of the project.

14.3 Reflection

AI-assisted development shifts the bottleneck from "can I implement this?" to "should I implement this now?" Shipping features quickly made scope management the hardest discipline. AI is a force multiplier for execution, but architecture, debugging, and constraint navigation are still fundamentally human work.

15 Resources & Links

Resource	URL
Live Application	https://oghmanotes.ie
GitHub Repository	https://github.com/semyonfox/oghma
Canvas LMS API Docs	https://canvas.instructure.com/doc/api/
Cohere Embed API	https://docs.cohere.com/reference/embed
pgvector Documentation	https://github.com/pgvector/pgvector
Auth.js Documentation	https://authjs.dev
AWS Amplify Gen 2	https://docs.amplify.aws
Next.js Documentation	https://nextjs.org/docs

Table 4: Project Resources and References